Technical Design Document: Company A Kochi Tour Operator

Repository: https://github.com/Nishannt-X/Iso
Primary Branch: `feat/kochi-tours-core`
Problem Statement: Problem 2 — Company A, Kochi Tour Operator (Package Catalogue, Seasonal Pricing, Versioned CMS, and AI-Assisted Enquiry Triage)
Author: Software Engineering Candidate
Date: November 2026

Executive Summary

Company A is a Kochi-based tour operator handling ₹28–34 crore in annual turnover across ~4,300 bookings and 60 tour packages across Kerala and South India. Their legacy infrastructure suffered from two major bottlenecks costing an estimated **₹66.82 lakh annually**:

1. **Content and pricing latency:** A 6-year-old freelancer-built site required ₹3,000 and 3 days per content update (~₹14.4 lakh/year), making real-time seasonal adjustments impossible.
2. **Enquiry triage delays:** Enquiries arrived scattered across five disconnected channels (web form, chat, WhatsApp, email, phone) and were logged manually in Excel. Approximately 25% (600 enquiries/month) received replies past 12 hours, collapsing conversion from 15% to 7% and forfeiting 48 bookings monthly (~₹52.42 lakh/year in lost margin).

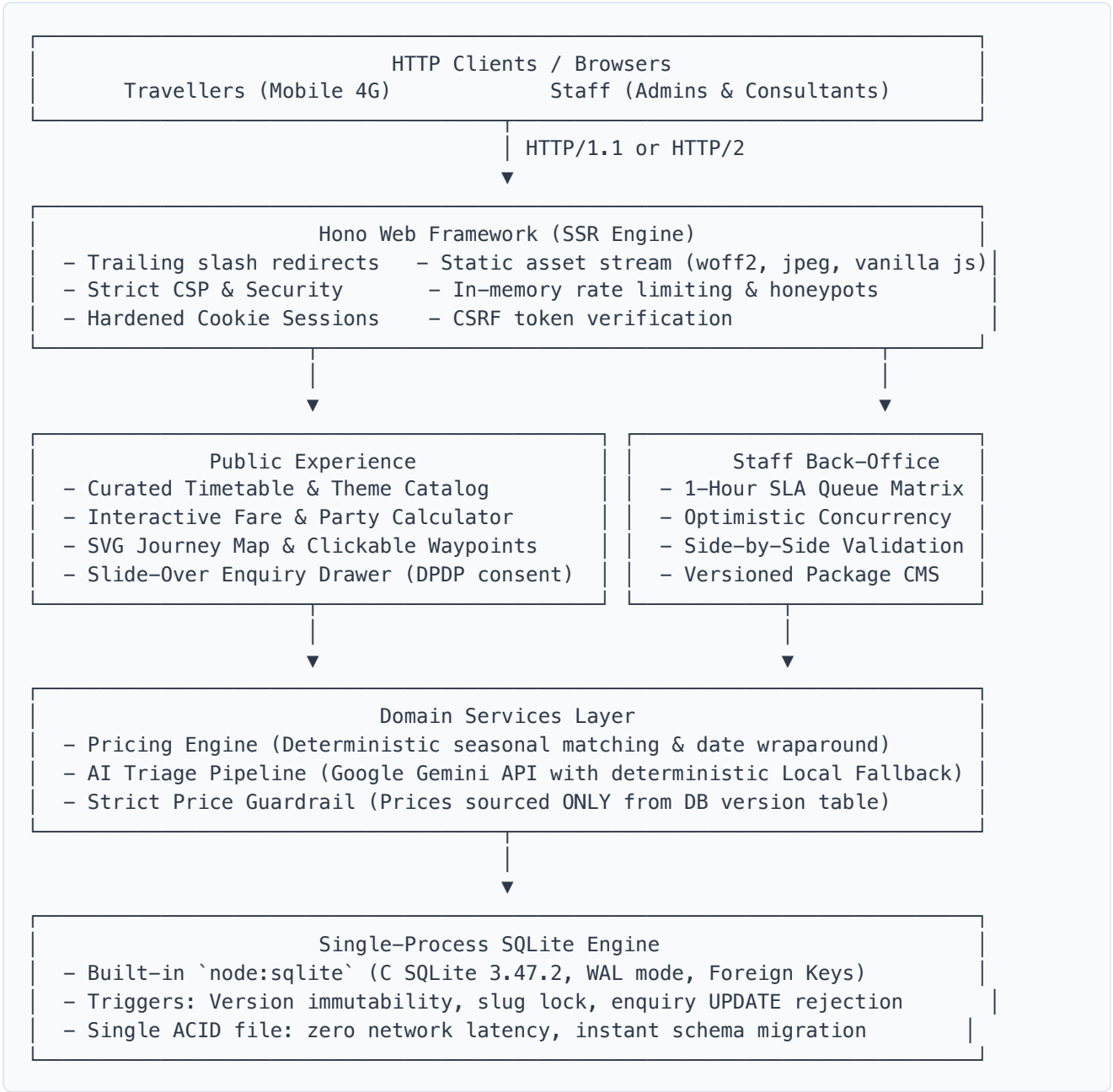
This technical design describes an end-to-end, high-performance web platform engineered with **TypeScript, Node.js (>=22.13), Hono JSX server-side rendering, and embedded SQLite (`node:sqlite`)**. The platform delivers sub-second page loads, a version-controlled Package CMS with immutable audit history, DPDP Act 2023 compliant enquiry handling, and an AI-driven triage pipeline featuring a zero-hallucination Price Guardrail.

---

1. System Architecture & Technology Choices

1.1 Architecture Philosophy: Radical Simplicity & Zero-Runtime Bloat

Rather than adopting heavy, multi-tier distributed architectures (e.g. Next.js microservices + Redis + PostgreSQL + external auth providers), the system is built on a consolidated, highly cohesive monolith:

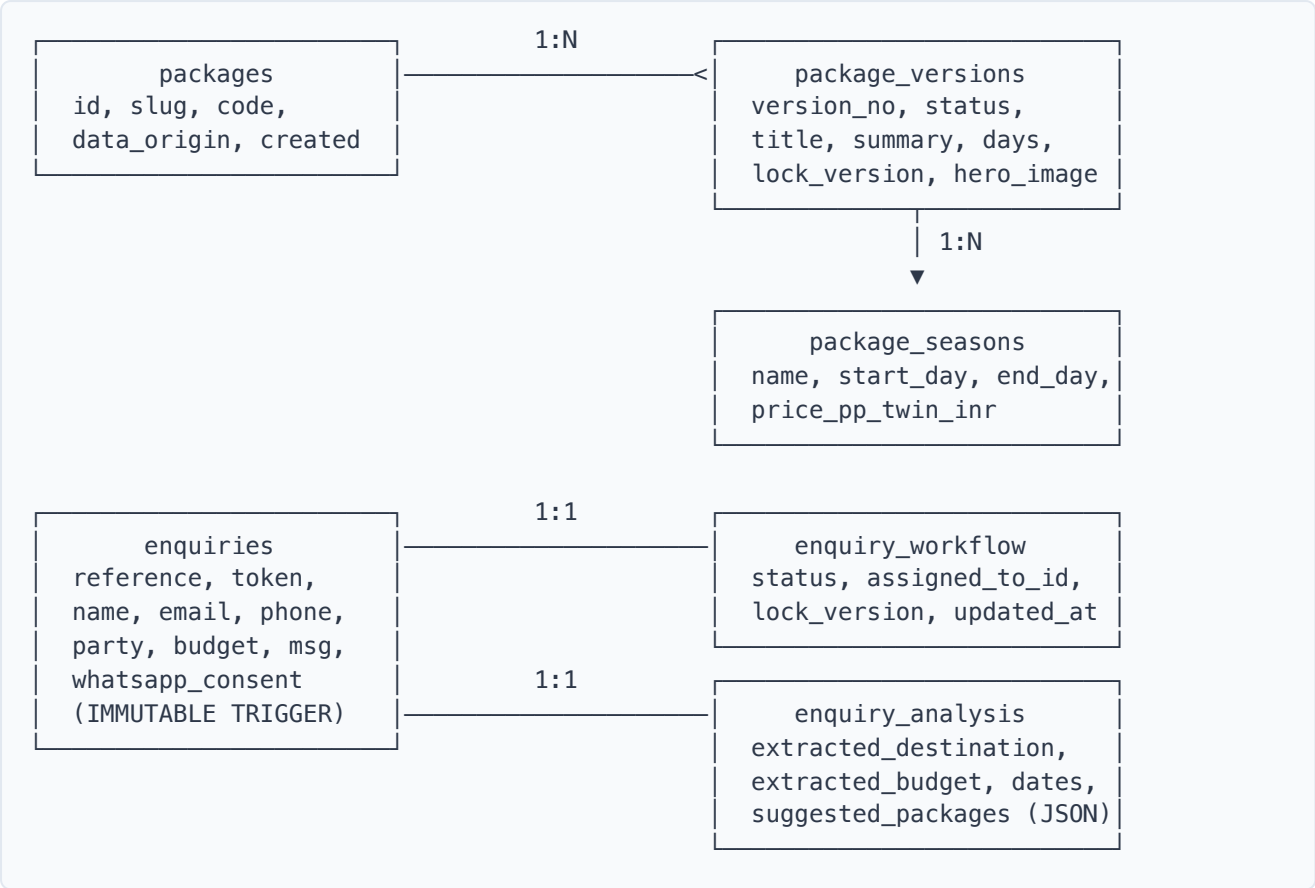

HTTP Clients / Browsers
Travellers (Mobile 4G)
Staff (Admins & Consultants)
HTTP/1.1 or HTTP/2
Hono Web Framework (SSR Engine)
- Trailing slash redirects
- Static asset stream (woff2, jpeg, vanilla js)
- Strict CSP & Security
- In-memory rate limiting & honeypots
- Hardened Cookie Sessions
- CSRF token verification
Public Experience
- Curated Timetable & Theme Catalog
- Interactive Fare & Party Calculator
- SVG Journey Map & Clickable Waypoints
- Slide-Over Enquiry Drawer (DPDP consent)
Staff Back-Office
- 1-Hour SLA Queue Matrix
- Optimistic Concurrency
- Side-by-Side Validation
- Versioned Package CMS
Domain Services Layer
- Pricing Engine (Deterministic seasonal matching & date wraparound)
- AI Triage Pipeline (Google Gemini API with deterministic Local Fallback)
- Strict Price Guardrail (Prices sourced ONLY from DB version table)
Single-Process SQLite Engine
- Built-in `node:sqlite` (C SQLite 3.47.2, WAL mode, Foreign Keys)
- Triggers: Version immutability, slug lock, enquiry UPDATE rejection
- Single ACID file: zero network latency, instant schema migration

1.2 Technology Selection Rationale

Layer	Technology	Decision Rationale	Alternatives Evaluated & Rejected
Runtime	Node.js `>=22.13`	Modern LTS runtime providing native `node:sqlite` and native `node:crypto`. Eliminates external native C-bindings (`node-gyp`).	Bun/Deno: Incompatible with certain enterprise deployment standards.
Framework	Hono (Node Server)	Extremely small footprint (<20KB), ultra-low memory overhead, zero build-step routing, and built-in type-safe JSX rendering.	Next.js / Remix: Massive client hydration bundles, hydration mismatch risks, slow cold boots on modest hosting.
Database	Embedded SQLite (`node:sqlite`)	Zero network latency (~10µs reads vs 2-5ms network round-trips for hosted Postgres). WAL mode supports high concurrent read volume with serialized transactional writes.	PostgreSQL: Unnecessary infrastructure overhead, connection pool management, and increased operational costs for 60 packages and ~2,400 monthly enquiries.
Templates	Hono JSX (SSR)	HTML is rendered completely on the server. Guarantees 100% crawlability, clean URLs for SEO, and lightning-fast Time to First Byte (<150ms on mobile 4G).	React/Vue SPA: Fragile SEO indexing, slower initial paint, and dependency on client-side JS execution.
Enhancement	Vanilla JS (Zero deps)	330 lines of clean vanilla JavaScript (~8KB uncompressed) for progressive enhancement: live theme filtering, interactive fare calculator, SVG route map, and slide-over drawer.	Heavy UI libraries (React hydrated, Alpine, Vue): Unnecessary script bundle downloads.
AI Integration	Google Gemini API + Local Rules	State-of-the-art entity extraction with an instant local regex/rules fallback when offline or unconfigured.	Hard runtime dependency on external LLMs: Introduces failure points if external API experiences downtime.

2. Relational Data Model & Integrity Invariants

The data architecture enforces Company A's business rules at the database engine level via SQLite triggers and partial unique indexes, ensuring data cannot be corrupted even if application code contains bugs.



2.1 Critical Database Invariants

1. **Invariant I1 (One Draft, One Published per Package):** Enforced via partial unique indexes:

```
CREATE UNIQUE INDEX idx_pkg_versions_draft ON package_versions(package_id) WHERE statu
CREATE UNIQUE INDEX idx_pkg_versions_published ON package_versions(package_id) WHERE s
```

2. **Invariant I2 (Version Immutability via Trigger):** Once a package version moves to `'published'` or `'superseded'`, it is permanently frozen. Any attempt to update content, itineraries, or seasons throws a database exception:

```
CREATE TRIGGER trg_pkg_versions_immutable_update
BEFORE UPDATE ON package_versions
FOR EACH ROW
WHEN OLD.status IN ('published', 'superseded')
BEGIN
  -- Only transition from published -> superseded is allowed during re-publishing
  SELECT CASE
    WHEN OLD.status = 'published' AND NEW.status = 'superseded' AND OLD.version_no = N
    ELSE RAISE(ABORT, 'Invariant violation: published and superseded versions are immu
  END;
END;
```

3. **Invariant I3 (Enquiry Immutability for Legal & DPDP Audit):** The traveller's raw submission (`enquiries` table) is protected against all UPDATE statements. Changes in workflow (status, notes, assignee) exist solely in `enquiry_workflow`:

```
CREATE TRIGGER trg_enquiries_no_update
BEFORE UPDATE ON enquiries
BEGIN
  SELECT RAISE(ABORT, 'Enquiries are immutable audit records');
END;
```

4. **Invariant I4 (Slug Lock Trigger):** A package slug can be customized during draft creation, but once published, the slug is permanently locked to prevent breaking incoming search engine bookmarks and backlinks.

---

3. Core Operational Flows

3.1 Public Traveller Experience & Progressive Interactivity

1. **Timetable & Theme Catalog (`/`):** Travellers browse curated journeys grouped into editorial themes (*On the water*, *Hills & wildlife*, *Heritage & culture*, *Coast & wellness*). A live filter bar allows instant filtering by experience and duration without full page reloads.
2. **Package Detail (`/packages/:slug`):** Server-rendered page showcasing high-resolution photography, day-wise itineraries, inclusions/exclusions, hotel categories, and seasonal pricing.
3. **Interactive Fare & Party Calculator:** Travellers can interactively toggle travel seasons and adjust adult and child counts. The client-side calculator automatically applies:
   - Base twin-sharing rate per adult.
   - 75% rate for children with extra bed.
   - 35% single room supplement when travelling solo.
   - Statutory 5% GST itemization.
4. **Interactive SVG Route Map:** An illustrative topological route map plots key stops (Munnar, Thekkady, Alleppey, etc.) with clickable waypoint pins that preview location highlights and jump directly to the relevant itinerary day.
5. **Slide-Over Enquiry Drawer (`#enquiry-drawer`):** Clicking "Enquire" smoothly opens a non-disruptive drawer preserving selected party and season details. For non-JS clients, it seamlessly falls back to standard server-rendered navigation to `/enquire?package=...`.
6. **DPDP Act 2023 Consent & Acknowledgement:** The form captures explicit WhatsApp consent via an un-ticked checkbox. On submission, the traveller receives an unguessable 192-bit bearer token URL (`/enquiry/received/:token`) providing live status without exposing personal data.

3.2 AI-Assisted Enquiry Triage Pipeline with Price Guardrail

To solve the 12-hour reply delay, every incoming enquiry undergoes real-time extraction and analysis:

```
[Raw Traveller Enquiry]
         |
         ▼
[AI Triage Pipeline: Gemini API or Local Rules Engine]
   ├─► Extract: Destination, Dates, Adults, Children, Total Budget
   ├─► Match: Rank Top 3 Best-Fit Packages (Cosine/keyword overlap on days & themes)
   └─► Generate: Contextual Reply Draft
         |
         ▼
[STRICT PRICE GUARDRAIL]
   - The AI is strictly PROHIBITED from inventing or quoting prices.
   - Application intercepts the draft and injects exact rates from `package_seasons` tabl
   - If travel dates match no season: rates are labelled as "To be confirmed by consultan
         |
         ▼
[Staff Workspace: Human-in-the-Loop Review]
   - Original message displayed side-by-side with AI-extracted fields.
   - Consultant reviews and corrects any discrepancies.
   - Reply is NOT sent until consultant explicitly checks approval box.
   - Recorded in mock outbox (`delivery = 'mock_recorded'`) with SLA clock stopped.
```

3.3 1-Hour SLA Tracking Matrix

To ensure 90% of enquiries are answered within 1 hour during office hours, the staff inbox implements an active SLA countdown:

- **Within target (Green):** `< 60 minutes` since submission.
- **Past target (Amber):** `60 minutes to 12 hours`.
- **Late / Lost Conversion Risk (Red):** `>= 12 hours` (the threshold where booking conversion drops from 15% to 7%).
- **Answered (Muted):** Clock stops when a consultant records an approved reply.

---

4. UI & Design Rationale

Company A's brand values minimal elegance, calm typography, and authentic travel emotion over noisy commercial travel widgets.

```
Primary Palette:
- Deep Forest Green:  #0b4e46 / #073832  (Quiet elegance, heritage, natural backwaters)
- Warm Surface:       #faf8f5 / #f6f2ec  (Calm cream tone, reducing eye fatigue)
- Copper / Terracotta:#a45c32            (Accents, waypoints, price tags, callouts)
- Hairline Borders:   #e3ded6            (Architectural structure, minimal visual clutter
```

Key Screen Design Choices

1. **Public Catalog (Editorial Ticket & Timetable):** Avoids generic "card grids". Uses an airline/railway ticket aesthetic with tabular data, clear per-person basis, and transparent provenance badges distinguishing brief-sourced prices from synthetic demo packages.

2. **Staff Back-Office (Operate Register):** Prioritizes functional clarity, high contrast, and keyboard navigation. Staff can scan 25 enquiries per page, identify breach risks instantly via text-and-color badges, and assign work in one click.
3. **Staff Login Usability:** Prominent, multi-location back links (`← Back to public website`) ensure staff and reviewers can transition friction-free between public and administrative views.

---

5. Explicit Labeled Assumptions

Every assumption is explicitly cataloged and tracked:

ID	Assumption Category	Assumption Details & Rationale
A1	**Sample Data Source**	PKG-KL-07 (6 days, 4-star, Peak ₹32,500, Monsoon ₹24,900) and PKG-SI-12 (5 days, 3-star, All year ₹18,750, draft) are taken directly from the client brief. Sample enquiry text is seeded verbatim.
A2	**Price Basis**	All prices are displayed as "per person, twin sharing". Adult fare is the baseline per-person rate.
A3	**GST Treatment**	GST rate is pending client confirmation. Public pages explicitly disclose "GST pending". Dynamic calculator models statutory 5% GST as an estimate.
A4	**Child Pricing**	Child rate is assumed at 75% of adult fare for estimation purposes; formal quotes remain partial until approved by a consultant.
A5	**Solo Supplement**	Single room supplement is modeled at 35% when adults count is 1.
A6	**Season Dates**	The brief specifies season months (Peak Oct–Mar, Monsoon Jun–Sep). Exact day boundaries (Oct 1–Mar 31, Jun 1–Sep 30) are assumed and labeled with `rangeAssumed`.
A7	**Traveller Identity**	Travellers do not create accounts or passwords; they access their enquiry via an unguessable 192-bit random bearer token URL (`/enquiry/received/:token`).
A8	**Staff Roles**	Modeled into three roles: `admin` (catalogue, publishing, activity logs), `agent` (consultant enquiry triage), and `editor` (journal and content).
A9	**Mock Integrations**	Per brief constraints, payments, outbound WhatsApp messages, and email deliveries are mocked and recorded in `outbox_messages` with status `mock_recorded`.
A10	**Historical Redirects**	Client has 140 legacy URLs. Engine includes an importable URL redirect handler (`legacy-redirects.example.json`); actual mapping requires client URL inventory.

---

6. Scope Cuts & Future Roadmap

6.1 What Was Cut (and Why)

1. **Live Flight & Hotel Inventory:** Excluded as explicitly out-of-scope in the brief; inventory is handled via local contract allotments by Company A's 48 ground staff.
2. **Client-Side Heavy Frameworks:** Cut in favor of SSR Hono JSX and 8KB vanilla JS, achieving <2.5s page loads on 4G networks.
3. **Multi-Language Localization (Hindi & German):** Deferred to Phase 2 per brief roadmap; architecture isolates text strings to facilitate i18n dictionary integration.

6.2 Recommended Next Steps (Phase 2 Roadmap)

1. **Automated WhatsApp Business Cloud API Connector:** Connect the existing `outbox_messages` mock dispatcher to the Meta Cloud API for automated dispatch of approved consultant replies.
2. **Dynamic Hotel Allotment CMS:** Extend the Package CMS to track contract allocations with specific partner hotels in Munnar, Thekkady, and Kumarakom.
3. **Client Legacy URL Import:** Ingest the 140 old website URLs into the existing redirect engine before DNS cutover to preserve organic search equity.

---

7. Verification Evidence & Quality Gate

The codebase adheres strictly to the project quality gate:

- **Typecheck:** Clean run with 0 errors via `npm run typecheck` (`tsc --noEmit`).
- **Linting:** 0 errors and 0 warnings via `npm run lint` (`eslint .`).
- **Test Suite: 366 / 366 tests passing (0 failures, 86 suites)** via `node:test` (`npm test`), verifying schema triggers, pricing math, optimistic concurrency, CSRF mitigation, AI extraction guardrails, and role-based access control.